Claude bilan ishlash

Amaliy qo'llanma va lifehack'lar.
Hujjatlardan emas, real tajribadan.

Format
7 bo'lim

Yo'naltirilgan
Workflow va amaliyot

Versiya
v1 · 2026-05-21

Mundarija

> Bu qo'llanma Anthropic hujjatining qayta yozilishi emas. Bu — Claude bilan kunda ishlovchi senior dev'ning real yo'l xaritasi: qaysi tool qachon, qanday promt yozish kerak, qaysi xatolarga yo'l qo'ymaslik va qaysi lifehack'lar vaqtni haqiqatan ham tejaydi.

01 BO'LIM · ASOSLAR

Claude imkoniyatlari va platformalari

✓ Claude imkoniyatlari

Imkoniyat	Nima qiladi	Natija
Design	Aytgan g'oyangizdan prototip, slayd yoki landing chiqaradi. Loyiha brand ranglari va token'larini o'qib moslay oladi.	PPTX, PDF, HTML, Canva, Code'ga handoff
Cowork	Coding emas vazifalarni o'zi bajaradi: fayllarni tartibga soladi, hisobot tuzadi, Excel'da analiz qiladi, browser orqali ko'p qadamli ish ko'radi.	Tayyor fayl (docx, xlsx, pdf), shell action
Code	Loyiha fayllarini o'qiydi va tahrir qiladi, terminal'dan komanda yuradi, git bilan ishlaydi, agent va hook chaqiradi.	Kod o'zgarishi, git commit
Connector (MCP)	Tashqi tizimlar bilan to'g'ridan-to'g'ri ulanadi: GitHub, Jira, Notion, Linear, Postgres, Slack. O'qiydi va o'zgartiradi.	Tashqi tizim ichida action
Skill	Murakkab vazifaning tayyor "retsepti": nima qilish, qanday tartibda, qaysi script kerak. Anthropic tomonidan yoki o'zingiz yozgan.	Skill bajargan natija

Claude platformalari

Platforma	O'ziga xos imkoniyat	Umumiy imkoniyat	Eng yaxshi qachon
Web claude.ai	**Design**	Chat, Connector, Skill	Tezkor chat, brainstorm, hujjat o'qish, prototip chizdirish
Desktop Mac/Win/Linux	**Cowork**, **Code** (grafik), lokal fayl access	Chat, Connector, Skill	Kunlik ish, multi-day refactor, lokal fayllar bilan ish
Mobile iOS/Android	Voice input, **Cowork'ga dispatch** yuborish	Chat, Connector (cheklangan)	Yo'lda fikr yozish, vazifani Cowork'ga uloqtirib qo'yish
CLI Claude Code	Hook, custom agent, headless rejim, terminal automation	Code, Connector, Skill	Git workflow, CI integration, scriptable ish

Desktop Code va Terminal CLI farqlari

Ikkalasi ham bir xil Code imkoniyatini ochadi, lekin amalda farq sezilarli:

Tomoni	Desktop Code	Terminal CLI
Interface	Grafik — sidebar, tab, vizual diff	Faqat terminal
O'tgan sessiyalar	Sidebar'da hamma sessiyalar ro'yxati (cheklov — disk hajmi). Har birining sarlavhasi va vaqti ko'rinadi. Click bilan butun suhbatni ochib, oldingi xabarlarni o'qish, kontekstni eslab olish oson.	`claude --resume` oxirgi sessiyalar ro'yxatini terminalda ko'rsatadi, strelka bilan tanlanadi. Sessiyalar `~/.claude/projects/<loyiha>/` da JSON sifatida saqlanadi. Oldingi xabarlarni o'qish uchun shu faylni ochish yoki sessiya ichidagi scrollback'dan foydalanish kerak.
Hook, custom agent	Sodda variantlar	To'liq qo'llab-quvvatlanadi
Headless rejim	Yo'q	Bor (CI, script, automation uchun)
Eng mos vaziyat	Multi-day refactor, vizual diff kerak bo'lganda, oldingi suhbatlarni tez-tez qayta ko'rib chiqishda	Git workflow, agent zanjirlari, scriptable ish, headless CI

02 BO'LIM · ASOSLAR

Til va so'rov sifati

⚠ Muammo

Claude inglizcha promtni eng oson tushunadi, lekin o'zbek tilini (lotin va kirill) ham bemalol qabul qiladi. Hatto ingliz, rus va o'zbek aralashgan promtlarni ham qiynalmasdan ishlaydi — fikrlash tabiiy oqimda kechishi mumkin. Asosiy savol — qaysi tilda yozish emas, balki *so'rovning qanchalik aniqligi*.

✓ Yechim — uchta qatlam

Qatlam	Savol	Maqsad
Niyat	Nima qilmoqchisiz?	"DataFrame tuzatish" emas, "CSV'ni dashboard'ga tayyorlash"
Kontekst	Qanday muhitda?	Python 3.11, pandas 2.x, NaN'lar bor
Mezon	Qanaqa natija mos?	5000 qator, 200ms ichida, memory < 500MB

▸ Misol PYTHON

YOMON — NIYAT NOANIQ

```
shu CSV ni parse qilib ber
```

YAXSHI — UCHCHALA QATLAM BOR

```
Bu CSV (rejalashtirilgan: 50K qator/oy) dashboard uchun kerak.
Muhit: Python 3.11, pandas 2.2. Datetime UTC bo'lishi shart.

Vazifa:
- date_str kolonkasini datetime'ga aylantir (format: 2026-03-14T09:30:00Z)
- NaN qiymatlar status='unknown' bo'lsin
- amount kolonkasini float64'ga; "$" va "," tozalash kerak

Natija: bitta funksiya `parse_orders(path: Path) -> pd.DataFrame`,
type-hinted, 200ms da 10K qatorni o'g'irlasin (pytest-benchmark bilan tekshiraman).
```

LIFEHACK

O'zbekcha aralash promt — bemalol. "DataFrame'ni groupby qilib, har bir `user_id` uchun median `session_duration` chiqar" — Claude buni hech aralashtirmaydi. Technical jargon (DataFrame, groupby, median) inglizcha qoladi, bog'lovchi so'zlar va niyat o'zbekcha. Bu — tabiiy yo'l.

TIPIK XATO

"Yaxshiroq yozib ber", "to'liqroq qil", "professionalroq bo'lsin" — bu mezon emas. Claude "yaxshiroq" ni o'zicha talqin qiladi. Aniq metrik bering: "200 satrdan oshmasin", "PEP 8 mos", "test coverage 80%+".

03 BO'LIM · SIFAT

PRIME va meta-prompting

⚠ Muammo

Promtning sifati javobning sifatini belgilaydi. Yetarli aniqlik berilmagan promt ikki xavfni keltiradi: javob mos chiqmaydi, qayta-qayta yozib aniqlashtirishga vaqt ketadi — yoki tuzatishga erinib eng yaxshi bo'lmagan javobni qabul qilib qolinadi. Ikkalasi ham yo'qotish. Yechim — promtni boshidanoq to'g'ri tuzish.

✓ Yechim — PRIME formati

P.R.I.M.E. — strukturali prompt yozish freymvorki, har bir harf quyidagini bildiradi:

Harf	Komponent	Tipik mazmun
P	Purpose — vazifa maqsadi va kerakli natija	"Order cancel API endpoint chiqarish — foydalanuvchi bekor qila olsin, refund avtomatik ishlasin"
R	Requirements — majburiy talablar, cheklovlar, kiritiladigan ma'lumotlar	"Loyihada `BaseController`, `@Validated`, GlobalExceptionHandler ishlatiladi. OpenAPI'ga mos"
I	Instructions — bajarish qadamlari (tartiblangan)	"1. PaymentController yarat. 2. POST endpoint qo'sh. 3. service chaqir. 4. exception throw qilma"
M	Metrics — o'lchov mezonlari (uzunlik, til darajasi, format)	"Bitta fayl, < 150 satr, javadoc yo'q, naming PascalCase, til: ingliz commit message"
E	Examples — misollar va namunalar	"`UserController` bilan solishtir — naming, return type, exception handling shu uslubda"

Qachon qaysi versiya

Vaziyat	Format	Vaqt
Yangi murakkab feature	To'liq **PRIME**	5-10 daq
Mavjud kod refactor (kontekst codebase'da)	**PRI** — M va E default'da qoladi	3-5 daq
Bir martalik script yoki kichik fix	**PI** — faqat maqsad va qadamlar	1 daq
Boshqa kishi uchun deliverable	**PRIME** — Examples bir nechta	10+ daq

▸ Misol JAVA

PRIME — SPRING BOOT CONTROLLER

```
P: Order cancel API endpoint chiqarish — foydalanuvchi orderni bekor
   qila olsin, refund avtomatik ishlasin, status SOFT_DELETED bo'lsin.

R: Loyihada BaseController, @RequestMapping("/api/v1"), GlobalExceptionHandler
   bor. PaymentService va PaymentDto allaqachon mavjud (Lombok @Data).
   OpenAPI'ga mos.

I: 1. PaymentController yarat (extends BaseController)
   2. POST /api/v1/payments/{id}/refund endpoint qo'sh
   3. PaymentService.refund(id) chaqir
   4. Exception throw qilma — service throw qiladi, handler ushlaydi
   5. Response: PaymentRefundDto qaytar

M: Bitta fayl, < 150 satr. Javadoc yo'q. @Validated majburiy.
   Naming: PascalCase. Til: ingliz commit message.

E: src/main/java/.../controller/OrderController.java bilan solishtir —
   naming, return type, exception handling shu uslubda bo'lsin.
```

Meta-prompting — Claude o'ziga promt yozadi

Boshlovchining eng katta vaqt tejovchisi. "Menga shu vazifa uchun promt yoz" deb so'rasangiz — Claude detallarni o'zi to'ldiradi va kerakli savollarni beradi.

TIPIK HOLAT

```
Figma'da landing page kerak.
Nima yozay?
```

META-PROMT

```
Menga Figma Make uchun promt yoz.
Landing — B2B SaaS, finance.
Hero, 3 feature, pricing, CTA.
Brand: navy + amber, sans-serif.
Sen mendan kerakli savol so'ra.
```

Claude buni Midjourney, Sora, ChatGPT, Cursor uchun ham qila oladi. Har AI ning o'z promt sintaksisi bor — Claude tarjima qiladi.

LIFEHACK

Murakkab promt yozish 10 daqiqa — ma'qul, lekin uni keyingi safar qayta yozmang. `~/.claude/templates/` papkasiga saqlang va kerak bo'lganda nusxalang. Yoki bir necha gal ishlatadigan promtni — o'z slash buyrug'ingiz qiling: `~/.claude/commands/` papkasiga `.md` fayl yozsangiz, CLI'da `/buyruq` deb chaqirib ishlatasiz.

Yomon va yaxshi promt — 5 misol

Chap tomondagi promtlar bir qarashda yetarli ko'rinadi — ko'pchilik shunday yozadi. Lekin har birida kritik aniqlik yetishmaydi va Claude taxminga majbur bo'ladi. O'ng tomon — shu yashirin kamchiliklarni yopadi.

1 · YETARLI EMAS

```
UserService.login() da NPE chiqyapti,
tuzat. Mavjud kodga qarab uslubni saqla.
```

1 · ANIQ

```
UserService.login() da NPE — user.getRole()
null kelganda. Sabab: yangi OAuth user'larda
role kelmaydi. Default USER tayinla, lekin
DB'ga yozma — faqat runtime'da. Existing
GlobalExceptionHandler'ni o'zgartirma.
```

2 · YETARLI EMAS

```
Order create endpoint — POST /api/v1/orders.
Items va address qabul qil, orderId qaytar.
Validation qo'sh, BaseController'dan foydalan.
```

2 · ANIQ

```
POST /api/v1/orders — bor edi, faqat
idempotency qo'shaman. Header: Idempotency-Key.
Bir xil key 24 soat kelsa — eski javobni
qaytar, yangi order yaratma.
Redis'da saqlanadi (RedisTemplate bor).
```

3 · YETARLI EMAS

```
BookingCubit uchun unit test yoz.
Happy path va error case'larni qamra,
bloc_test framework bilan.
```

3 · ANIQ

```
BookingCubit.submitBooking() uchun test.
Faqat shu metod. Mock: BookingRepository.
Test: (1) success → BookingSuccess,
(2) 409 → BookingConflict,
(3) timeout → BookingError.
State ketma-ketligi: emitsInOrder bilan.
```

4 · YETARLI EMAS

```
OrderListView sekin ishlayapti,
performance'ni optimallashtir. List katta
bo'lganda lag bor.
```

4 · ANIQ

```
OrderListView 500+ element'da lag qiladi.
Profile qildim — har scroll'da butun list
rebuild. Sabab: parent setState. Faqat
ListView.builder + const item'ga o'tkaz,
parent state'ga tegma. Pagination QO'SHMA.
```

5 · YETARLI EMAS

```
Login'dan keyin token saqlanmayapti.
Restart'da yana login so'raydi. Token
storage logikasini tekshirib, tuzatib ber.
```

5 · ANIQ

```
Token restart'da yo'qoladi. Topdim:
secure_storage'ga yoziladi, lekin login'da
o'qilmaydi (faqat memory'dan). auth_bloc
init'da secure_storage'dan o'qishni qo'sh.
Migratsiya shart emas — yangi user'lar uchun.
```

*E'tibor bering: chap promtlar ham "yaxshi promt" kabi ko'rinadi — fayl, framework, hatto talab ham bor. Yetishmagani — **kontekst va chegara**: nega muammo chiqgani, nimaga tegmaslik kerakligi, qaysi alternativani tanlamaslik. Aynan shu Claude'ni keraksiz taxmin va ortiqcha o'zgarishdan saqlaydi.*

04 BO'LIM · SOZLASH

Global instructions — Claude sizning uslubingizni o'rganadi

Har yangi chat'da bir xil ma'lumotlarni qaytarib o'tirishga vaqt sarflanadi: "javob o'zbekcha bo'lsin", "qisqa yoz", "kod uchun jadval bilan ko'rsat". Agar yozmasangiz yoki har xabarda eslatib turmasangiz — Claude sizning uslubingizni bilmaydi, kutilgan javobni bera olmaydi. Yechim — global instructions'ni bir martalik aniq va batafsil yozish. Naqadar yaxshi to'ldirsangiz, shuncha kam tuzatish va eslatish kerak bo'ladi.

✓ Yechim — global instructions ikki joyda

Platforma	Joy	Kimga
Claude.ai web/desktop	Settings → Profile → "Instructions for Claude"	Chat, Design, Cowork ishlatuvchilar
Claude Code CLI	`~/.claude/CLAUDE.md`	Terminal'da ishlovchilar

Ikkalasi ham hamma sessiyaga qo'llaniladi. Mantiq bir xil — bir martalik yozasiz, hamisha amal qiladi. Eng yaxshi yondashuv — ikkalasiga ham yozish (universal qoidalar). Faqat CLI'ga xos narsalar (git, hook, headless rejim) — faqat `CLAUDE.md`'da.

Yaxshi global instructions tarkibiy qismlari

Qism	Nima yoziladi	Maqsad
ALWAYS	Har javobda majburiy qoidalar (til, qisqalik, format)	Claude bu qoidalarni hech qachon buzmaydi
CONTEXT	Siz kim, qaysi sohada, qaysi tool'da ishlaysiz	Javobni sizning darajangizga moslashtiradi
OUTPUT FORMAT	Javobning struktura modeli (masalan, STATUS block)	Har javob bir xil shaklda — audit oson
KEYWORDS	`@` prefix bilan har xabarda rejim almashtirish	Bitta xabar uchun maxsus rejim (qisqaroq, chuqurroq, tanqid)
CODE & DECISIONS	Kod yozish va qaror qabul qilish uslubi	Implement'ga sakramaslik, non-idiomatik patternni flag qilish

▸ Misol — STATUS block formati

Har javob oxirida bir xil tartibda 5 ta maydon — har biri bir qator:

```
OUTPUT FORMAT
ISHONCH — confidence darajasi (foiz + sabab)
OGOHLANTIRISH — taxmin, kamchilik, xavf. "yo'q" agar yo'q bo'lsa
MANBA — src:web, src:training, src:memory, src:file, src:user
TEKSHIRUV — nima tasdiqlangan, qaysi manba qaytadan o'qilgan
TAKLIF — keyingi qadam yoki tegishli takliflar. "yo'q" agar yo'q bo'lsa
```

Bu format javobni audit qilishni juda osonlashtiradi. Claude'ning ishonchini, manbasini va qoldirgan kamchiliklarini bir qarashda ko'rasiz.

STATUS block kim uchun foydali, kim uchun ortiqcha

✅ Foydali	⚠️ Ortiqcha bo'lishi mumkin
Yangi foydalanuvchi — Claude'ning ishonchi va manbasini bir qarashda ko'rib o'rganadi	Tezkor chat — har javobda 5 qator qo'shimcha, kerak bo'lmasa shovqin
Kritik vazifa — kod review, security audit, hisobot — har javobning kamchiliklarini auditga olish kerak	Oddiy savol-javob — "qaysi yil edi", "shu kod ishlaydimi" kabi qisqa savollar
Hujjat va deliverable — manba va ishonch qatorlari professional sifat ko'rsatadi	Casual brainstorm — fikr almashish paytida format formaliyat tug'diradi

Agar STATUS block ba'zi xabarlar uchun ortiqcha bo'lsa, KEYWORDS bilan almashtirish mumkin: `@ResultShort` bilan qisqa javob — STATUS block'siz yoki minimal versiyada chiqadi.

▸ Misol — KEYWORDS tizimi

Har xabarga prefix qo'shish orqali bitta xabarga ta'sir qiluvchi rejim almashtirish:

Guruh	Keyword	Ma'no
Accuracy	`@AccuracyStrict`	Faqat tasdiqlangan manba, taxmin yo'q
Accuracy	`@AccuracyLoose`	Inference va reasonable guess ruxsat
Result	`@ResultShort`	1-3 jumla, misol/jadval yo'q
Result	`@ResultFull`	To'liq tahlil, trade-off, misol
Mode	`@ModeBrainstorm`	Variantlar generatsiya, qaror yo'q
Mode	`@ModeAudit`	Read-only — faqat aniqlash, o'zgartirish yo'q
Effort	`@EffortQuick`	Tezkor javob, alternativa minimal
Effort	`@EffortDeep`	Chuqur tahlil, edge case, second-order ta'sir
Review	`@ReviewDebate`	Mening pozitsiyamga qarshi argument
Review	`@ReviewFindFlaw`	Zaiflik, xavf, risk topish
Review	`@ReviewPivot`	Tubdan boshqacha yondashuv taklif
Review	`@ReviewImprove`	Mening yondashuvimni saqlab yaxshilash

Kombinatsiya ruxsat: `@AccuracyStrict @ResultShort` yoki `@ModeBrainstorm @EffortDeep`. Konflikt qoidasi: Accuracy hamisha yutadi, qolganlari uchun — birinchi yozilgan keyword g'olib.

▸ Misol — Instruction for Claude (web/desktop)

```
SETTINGS → PROFILE → INSTRUCTIONS FOR CLAUDE

ALWAYS — every response, no exceptions:
1. Communicate in Uzbek (Latin script, straight apostrophe only — U+0027).
2. Be concise — no padding, no unsolicited caveats, no restating my question.
3. Use tables for comparisons and decisions.
4. Every response ends with one STATUS block.

CONTEXT:
Senior Android and Flutter Engineer. Uses Claude Code CLI and macOS desktop
Code section directly — no CLI prompts in chat.

OUTPUT FORMAT:
End every response with ONE status block (code block). Fields in this exact
order, always all present:
  ISHONCH — confidence (percent — short reason).
  OGOHLANTIRISH — assumption, gap, or risk. "yo'q" if none.
  MANBA — source. src:web, src:training, src:memory, src:user, src:file...
  TEKSHIRUV — what was verified or cross-checked.
  TAKLIF — next step or related suggestion. "yo'q" if none.

KEYWORDS (prefix any message with one or more, space-separated):
  Accuracy:  @AccuracyStrict, @AccuracyLoose
  Result:    @ResultShort, @ResultFull
  Mode:      @ModeBrainstorm, @ModeAudit
  Effort:    @EffortQuick, @EffortDeep
  Review:    @ReviewDebate, @ReviewFindFlaw, @ReviewPivot, @ReviewImprove

Conflict resolution:
- @AccuracyStrict always wins.
- For non-Accuracy conflicts: first keyword written wins.

CODE & DECISIONS:
- Don't jump to implementation. Ask clarifying questions when ambiguity high.
- Don't ask on every turn — prefer offering ("Savollarim bor, so'raymi?").
- Flag non-idiomatic patterns with better alternatives.
- Preserve ALL existing functionality during migration/refactor.
```

▶ Misol — `~/.claude/CLAUDE.md` (CLI)

~/.CLAUDE/CLAUDE.MD FRAGMENTI

```
## Communication
- Respond in Uzbek unless the user switches language
- Be concise — avoid unnecessary explanations for senior-level tasks
- When complex task has open questions, offer to ask first
- Don't ask about things you can determine by reading the code

## Model & Effort
- Default model: Opus. Agents override with own `model` field.
- Per-message override via @EffortQuick / @EffortDeep keywords.

## Agent Delegation
Available user agents (~/.claude/agents/):
- architect-review-agent  (opus)    — system design review
- flutter-ui-generator    (opus)    — new Flutter widgets/screens
- db-migration-agent      (opus)    — DB schema changes
- dto-generator-agent     (sonnet)  — DTOs from JSON/API
- git-review-agent        (opus)    — review git changes before push
- localization-agent      (sonnet)  — translation strings
- release-build-agent     (haiku)   — build/release commands

## Code Standards
- Read first, write second — check real codebase, never assume
- Flag non-idiomatic patterns with better alternatives
- Suggest improvements: naming, architecture, performance

## Logging & Comments
- Log messages in English
- Do not write code comments by default (user opted out)
- Acceptable: user asks explicitly, or genuinely necessary platform workaround

## File Operations
- Never create `*.md` files (READMEs, notes, status docs, plans)
- If documentation seems useful, say so inline — don't create file
- Stay within scope; don't reformat or "improve" unrelated code

## Git Rules
- NEVER commit, push, create branches, merge, rebase, or tag
- Always use `git mv` for rename/move (preserves blame history)
- Edit files in place — don't delete + recreate

## Migration & Refactoring
- Current implementation = source of truth
- After writing new code, compare old vs new line-by-line
- Prefer small focused changes over big-bang rewrites

## Terminal Commands
- Write commands without inline comments (for easy copy-paste)
- Write explanations separately, before or after the command
```

Tavsiya etiladi — yozish yaxshi narsalar

✅ Yozing	Nima uchun
Til, apostrof va imlo qoidalari	Claude javobni sizning til standartlaringizga moslaydi
Javob uzunligi va format (STATUS block)	Audit oson, format barqaror, kerakli ma'lumot yo'qolmaydi
KEYWORDS tizimi (`@` prefix)	Bitta xabarda rejim almashtirish — har gal qaytarmaslik
Code style va idiomatik patternlar	Javob sizning uslubingizga mos chiqadi
Git va migration qoidalari (CLI uchun)	Tasodifiy commit/push, blame history yo'qotish xavfini oldini olish
Aniq mezon va o'lchov (uzunlik, coverage %)	"Yaxshiroq bo'lsin" emas — Claude aniq ma'noda yozadi

Yozishga arzimaydi — zid juftliklari

❌ Yozmang	Nima uchun
Til o'rniga: "umuman normal yozsin"	Bo'sh qoida — Claude o'zicha talqin qiladi, har gal turlicha chiqadi
Format o'rniga: "yaxshi ko'rinishda bo'lsin"	Ehtimol Claude jadval, ehtimol matn yozadi — barqaror format yo'q
KEYWORDS o'rniga: har xabarda qoidani qaytarish	Vaqt yo'qotish — bir martalik kiritilsa, doim ishlaydi
Code style o'rniga: implementatsiya detali (Lombok @Data)	Tez-tez o'zgaradi — kodga qarab Claude o'zi ko'radi
API kalit, parol, mijoz nomi	Disk'da plain text — repo'ga tushib ketishi mumkin
Sprint kontekst ("hozir auth feature ustida")	2 hafta keyin eskirib qoladi — bu kontekst chat'da bo'lsin

> **LIFEHACK**
>
> Global instructions juda kuchli — lekin uzunligi ham muhim. 150-200 satrdan oshirmang. Aniqroq qoidalar (ma'lum stack uchun) — path-scoped rules'da bo'lsin (`~/.claude/rules/python.md`, `~/.claude/rules/react.md`). Frontmatter'da `paths:` ko'rsating, Claude faqat mos fayllarda ishlatadi. Bu — har vazifaga to'g'ri qoidani avtomatik yuklash.

05 BO'LIM · SOZLASH

Project knowledge va Project instructions

Loyiha o'sgan sayin Claude'ga uning standartlarini tushuntirish ham qiyinlashadi. Lekin endi global instructions (4-bo'lim) sozlangach, savol o'zgaradi: loyiha uchun yana nima yozish kerak? Javob — siz CLI ishlatasizmi yoki yo'qmi, shunga bog'liq. Global'da bor narsani loyiha darajasida takrorlash ortiqcha, lekin loyihaga xos narsalar baribir kerak.

✓ Tushuncha — ikki qatlamli kontekst

Loyiha haqida Claude'ga ma'lumot berishning ikki yo'li bor:

Qatlam	Nima qiladi	Hajm
Project Instructions	Loyihaga xos qoidalar — naming, architecture, do/don't. Har savolda Claude bularni avtomatik kontekstga oladi.	holatga bog'liq
Project Knowledge	Real ma'lumotlar — schema, OpenAPI spec, ER diagram, base class'lar manbasi. Claude kerak bo'lganda o'qiydi.	10-50 fayl

Project Instructions'ga nima yozish — Claude Code (CLI) ishlatishingizga bog'liq. Quyida ikki holat.

Holat 1 — CLI ishlatasiz (ko'pchilik senior dev)

Agar kodni asosan terminaldagi Claude Code'da yozdirsangiz, loyiha qoidalari `./CLAUDE.md` faylida bo'ladi — buni CLI o'qiydi. Lekin chat (web/desktop) bu faylni o'qimaydi: u faqat global "Instructions for Claude" va Project'ning o'z Project Instructions'ini ko'radi. Shuning uchun ikki kichik holatga e'tibor bering:

1.1 — Project Instructions

Agar shu loyiha bo'yicha bir vaqtning o'zida chatda ham (web/desktop) Project ochib ishlamoqchi bo'lsangiz — Project Instructions'ga qisqa, chatda so'raladigan narsalarnigina yozing:

```
PROJECT INSTRUCTIONS — CHAT UCHUN (QISQA)
# Mobile Booking App — chat kontekst

## Stack (qisqa)
- Flutter + Custom Cubit (BaseCubit), dio + retrofit, freezed DTO

## Naming
- Cubit: <Feature>Cubit · API request: <Action>Request
- DTO response: <Entity>Response · File: snake_case

## Kod yozishda
- Domain layer pure Dart (Flutter import yo'q)
- ARB localization, Uzbek faqat straight apostrofi (U+0027)
- Brand: app_colors.dart / app_theme.dart (inline color yo'q)

## Eslatma
- Git, terminal, fayl operatsiyalari — bu yerda yo'q (chat ularni qila olmaydi)
```

1.2 — Local `./CLAUDE.md`

Loyiha root'idagi `./CLAUDE.md` faylga faqat global `~/.claude/CLAUDE.md`'dan *farq qiladigan* yoki loyihaga xos qo'shimcha qoidalarnigina yozing. Global'da bor narsani (til, umumiy git qoidasi, comment siyosati) qaytarmang:

```
./CLAUDE.MD — LOYIHA ROOT (FAQAT FARQLAR)
# Project-specific overrides

## Stack
- State: Custom Cubit (BaseCubit<STATE, EVENT>) — Bloc emas
- Code gen: freezed + json_serializable (build_runner)

## Architecture
- Feature-based: features/<feature>/{data, domain, presentation}
- Yangi Cubit'dan oldin base_cubit.dart o'qi

## Loyihaga xos
- Brand token: app_colors.dart, spacing: 4/8/12/16/24/32
- SMS OTP: Android SMS Retriever API
- Bu loyihada test coverage 70%+ talab (global default emas)
```

Holat 2 — CLI ishlatmasangiz (faqat chat/web)

Agar faqat chatda yoki Desktop'da ishlasangiz, `./CLAUDE.md` yo'q. Chat global "Instructions for Claude"ni o'qiydi, lekin u butun loyihaga emas, hamma narsaga umumiy. Loyihaga xos qoidalar uchun yagona joy — Project Instructions. Shuning uchun bu yerda batafsil yozing, chunki bu Claude'ning loyiha haqidagi to'liq bilim manbai. Pastdagi to'liq namuna kabi — stack, architecture, naming, do/don't hammasini qamrang:

```
PROJECT INSTRUCTIONS — CHAT/WEB (BATAFSIL)
# Mobile Booking App — Project Instructions

## Stack
- Flutter 3.x, Dart 3.x
- State: Custom Cubit (BaseCubit, BaseListener, BaseBuilder)
- Networking: dio + retrofit (generated clients)
- DI: get_it · Code gen: freezed (DTO), json_serializable

## Architecture
- Feature-based folder: features/<feature>/{data, domain, presentation}
- Domain layer — pure Dart, Flutter import yo'q
- BaseCubit<STATE, EVENT> — har feature uchun
- Single-event emitter (emitEvent), state immutability (Equatable)

## Naming
- File: snake_case (booking_cubit.dart, document_upload_page.dart)
- Class: PascalCase (BookingCubit, DocumentUploadPage)
- Cubit: <Feature>Cubit · API request: <Action>Request
- DTO response: <Entity>Response

## DTO / API
- freezed + json_serializable, @JsonKey har field (backend snake_case)
- Nullable backend contract'ga qarab

## Localization / UI
```

```
- ARB: uz, ru, en · Key: <screen>_<element>_<action>
- Uzbek: faqat straight apostrofi (U+0027)
- Brand: app_colors.dart, spacing 4/8/12/16/24/32, inline color yo'q

## Kod yozishda do/don't
- Do: yangi Cubit'dan oldin base_cubit.dart o'qi, mavjud feature'ga qara
- Do: existing theme va spacing'dan foydalan
- Don't: generic helper yaratma — avval utils/ ni tekshir
- Don't: kod ichiga izoh (comment) yozma
```

Qoida va fakt — qaysisi qaerga

✅ Project Instructions ga	📁 Project Knowledge ga
Naming convention (file, class, cubit, request)	API endpoint full list (OpenAPI yaml)
Architecture qatlamlari va folder strukturasi	ER diagram, DB schema
Do/don't qoidalari (naming, struktura)	Domain glossary (Booking, Payment, Refund terms)
Brand ranglari va spacing scale	Base class'lar manbasi (base_cubit.dart matni)
Localization qoidalari (apostrof, key naming)	External API contracts (third-party servislar)

Sodda farq: **qoida** — Instructions, **fakt** — Knowledge.

Yozishga arzimaydi

❌ Yozmang	Nima uchun
1000+ satr DB schema Instructions'ga	Har savolda token sarflanadi — Knowledge'da fayl qiling
Sprint kontekst ("hozir auth feature ustida")	Project Instructions ham eskirib qoladi — kontekst chat'da bo'lsin
API keylar, parol, mijoz nomi	Repo'ga tushish xavfi — har doim secret manager
Implementatsiya detali ("dio versiya 5.4.0")	O'zgaradi tez-tez — kodga qarab Claude o'zi ko'radi

> **LIFEHACK**
>
> Qaror qoidasi — "har gal tekshirish kerak bo'lgan qoida" → Instructions. "Bir gal o'qib eslab qolish kerak bo'lgan ma'lumot" → Knowledge. Schema → Knowledge, "DTO ga freezed ishlat" → Instructions. Savol oddiy: *qoidami yoki faktmi?*

06 BO'LIM · WORKFLOW

Agentlar — kontekst va model strategiyasi

⚠ Muammo

Hamma vazifa main session'da bajariladi: arxitektura review ham, oddiy DTO generatsiya ham, release build ham. Natija — ikki muammo birvarakayiga. Birinchisi: **kontekst bloating** — Claude oldingi vazifalar kontekstini ko'tarib yuradi, asosiy diqqat susayadi. Ikkinchisi: **model misallocation** — Opus oddiy formatlashda yoqilgan turibdi, Haiku bir marta ham ishlatilmaydi. Pul va sekinlik.

✓ Yechim — agent delegatsiya 3 qatlamli savol

1. **Kontekst alohida bo'lishi foydalimi?** (vazifa main session'ga aralashmasligi kerakmi?)
2. **Model murakkabligi qanday?** (Haiku yetadi, Sonnet kerak, yoki Opus shart?)
3. **Output formati deterministikmi?** (har gal bir xil shaklda chiqishi kerakmi?)

Uchchalasiga "ha" javob bo'lsa — agent. Aks holda main session.

Model tanlash — har agent uchun

Model	Qachon	Tipik agent vazifalari
Haiku	Tez, deterministik, takrorlanuvchi	release-build, lint-fix, format, simple translation
Sonnet	O'rtacha murakkab, struktura aniq	dto-generator, localization-agent, test-stub, doc-comment
Opus	Tafakkur talab qiladi, kontekst boy	architect-review, security-audit, refactor-planner

▸ Misol REACT

```
~/.CLAUDE/AGENTS/COMPONENT-REVIEW-AGENT.MD
---
name: component-review-agent
description: Yangi React komponentini jamoa standartlariga solishtirib tekshirish.
  Ishlatish: yangi komponent yaratilganda yoki katta refactor'dan keyin.
  Ishlatmaslik: bug-fix yoki bir satrlik o'zgarish uchun.
model: opus
tools: [Read, Grep, Glob]
---

Sen senior React reviewer'san. Quyidagilar tekshiriladi:

1. **Naming** — PascalCase fayl, default export bilan bir xil
2. **Structure** — Props interface yuqorida, hooks tartibda (state → effect → memo)
3. **Performance** — kerak bo'lmagan re-render, useMemo/useCallback noto'g'ri ishlatish
4. **A11y** — alt text, aria-label, semantic HTML
5. **Test** — har komponent uchun *.test.tsx bo'lishi shart

Faqat aniqlanadi, o'zgartirilmaydi. Har topilgan muammoni
file:line ko'rinishida ber, severity (critical/warning/info) bilan.
```

Agent description — eng kam baholangan qism

Claude qaysi agent'ni ishga tushirishni description'dan o'qib qaror qiladi. Description'da ikkita qator majburiy: **"Ishlatish: ..."** va **"Ishlatmaslik: ..."**. Aks holda Claude noto'g'ri agent'ni tanlaydi yoki agent'ni umuman e'tiborga olmaydi.

> LIFEHACK
>
> **Agent — context isolation tool, hatto bitta dev uchun ham.** Refactor o'rtasida "hozir security audit kerak edi" deb yoqlanganda main session'ni buzmang. `security-audit-agent` ishga tushiring — natija oling — main session arxitektura ishini davom ettiradi. Bu — focus, nafaqat performance.

> TIPIK XATO
>
> Har agent uchun default'ga `model: opus` qo'yish. Kichik vazifalar (lint, build, format) Haiku'da 5x tez va 10x arzon bajariladi. Opus'ni faqat haqiqatan tafakkur kerakli joyga qoldiring.

07 BO'LIM · POWER-USER

CLI buyruqlar va @ keyword tizimi

⚠ Muammo

Ko'pchilik CLI'da chat qiladi va shu bilan to'xtab qoladi. Slash buyruqlar va keyword tizimini bilmasdan — Claude Code'ning 30% qobiliyatidan foydalanyapsiz, xolos.

✓ Yechim — built-in buyruqlar to'plami

Buyruq	Vazifa	Qachon
`/plan`	Bajarishdan oldin reja	Multi-file change, migration
`/goal`	Uzoq muddatli maqsad (sessiya bo'ylab eslaydi)	"Bu sprint — auth refactor"
`/model`	Modelni sessiya uchun o'zgartirish	Murakkab → Opus, oddiy → Haiku
`/effort`	Reasoning chuqurligi	Critical task — high; tezda — low
`/compact`	Kontekstni siqib, asosiyini saqlash	Sessiya cho'zilib ketganda
`/agents`	Mavjud agent'lar ro'yxati va invoke	Maxsus task uchun delegatsiya
`/plugin`	Plugin marketplace va install	Yangi tool qo'shish

@ keyword — har xabarda rejim almashtirish

Slash buyruqlar — sessiya darajasida. @ keyword — bitta xabarga ta'sir qiladi. Combinatsiya:

Guruh	Keyword'lar	Maqsad
Accuracy	`@AccuracyStrict` · `@AccuracyLoose`	Manba ishonchi darajasi
Result	`@ResultShort` · `@ResultFull`	Javob hajmi
Mode	`@ModeBrainstorm` · `@ModeAudit`	Yondashuv
Effort	`@EffortQuick` · `@EffortDeep`	Reasoning
Review	`@ReviewDebate` · `@ReviewFindFlaw` · `@ReviewPivot` · `@ReviewImprove`	Tanqid uslubi

▸ Misol PYTHON

```
/PLAN ISHLATISH — DB MIGRATION
/plan
auth modulida User model'ni o'zgartiraman:
- email_verified_at: DateTime kolonkasi qo'shilsin (nullable=False, default=now)
```

```
- soft_delete uchun deleted_at qo'shilsin
- existing user'lar uchun migration data fill kerak (email_verified_at = created_at)

Alembic ishlatamiz. Production'da 2M+ user, migration 30 sekunddan oshmasin.

Reja tuz, men ko'rib chiqaman, keyin /accept yoki o'zgartiraman.
```

Claude reja tuzadi (qaysi fayl, qanday tartib, rollback strategiya), kod yozmaydi. Siz ko'rasiz, tuzatasiz, yoki tasdiqlaysiz — keyin ijro qiladi.

> **LIFEHACK**
>
> **O'zingiz uchun slash buyruq yarating.** `~/.claude/commands/release-notes.md` — ichida sizning release notes template'ingiz. Endi `/release-notes` yozsangiz, Claude template'ni o'zi ishlatadi. Har gal qoidalarni qaytarmaslik — bir martalik investitsiya.